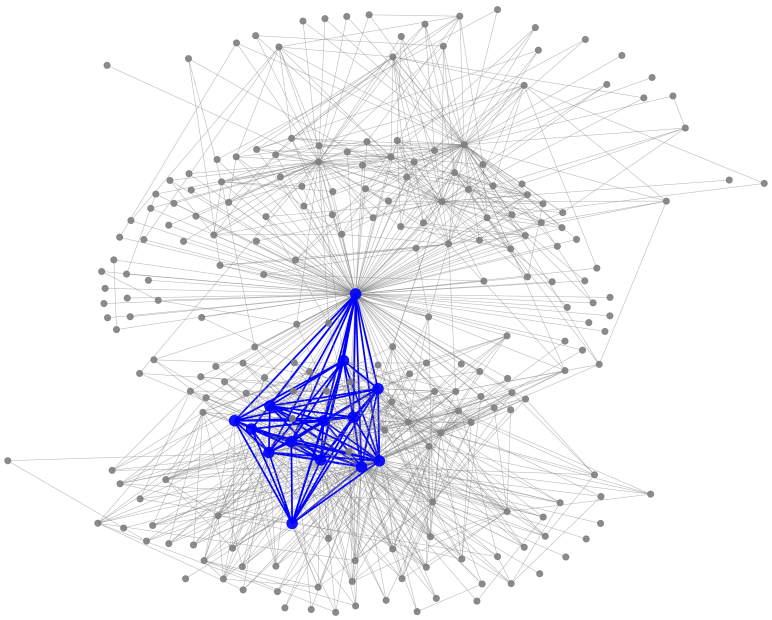
Erasmus University Rotterdam
Erasmus School of Economics
Bachelor Thesis Econometrics and Operations Research

DEEP LEARNING-GUIDED TABU SEARCH FOR THE MAXIMUM QUASI-CLIQUE PROBLEM

Tum Gommans (588362)



Supervisor:	Pedro José Correia Duarte
Second assessor:	Jehum Cho
Date final version:	5th July 2025

Abstract

The Maximum Quasi-Clique Problem (MQCP) involves finding the largest vertex subset of a graph that meets a specified density threshold. Due to the $\mathcal{NP}$-hard nature of the MQCP and its practical relevance across diverse fields including bioinformatics, social networks, and finance, efficient heuristics are essential. This thesis introduces `DeepTSQC`, a metaheuristic algorithm that extends the Tabu Search for Quasi-Cliques (`TSQC`) method (Djeddi et al., 2019). `DeepTSQC` utilizes structural graph features to predict optimal search parameters, significantly improving runtime performance while maintaining or enhancing solution quality. Comprehensive experiments on dense benchmark graphs and sparse real-life networks demonstrate `DeepTSQC`'s effectiveness, achieving speedups of 2–4 times over `TSQC`. Additionally, `DeepTSQC` occasionally outperforms `TSQC` regarding solution stability for sparse graphs at higher density thresholds. Despite the overhead of predictive modeling and challenges posed by imbalanced training data, the integration of Deep Learning into tabu search presents clear practical advantages.

1 Introduction

Given an undirected graph, the Maximum Quasi-Clique Problem (MQCP) aims to find a subgraph of maximum vertex cardinality, such that the density of the subgraph is at least a certain predefined threshold. Whether analyzing protein-protein interaction networks, bipartite item-user interactions on platforms like Spotify, or financial networks involving mutual credit arrangements, maximum quasi-cliques play a central role. They are, quite literally, everywhere. As real-world graph data is often noisy and incomplete, the MQCP is of particular practical importance as it generalizes the Maximum Clique Problem (MCP) by relaxing the full-connectivity requirement on the subgraph.

Tabu Search for Quasi-Cliques (`TSQC`) by Djeddi et al. (2019) constitutes a leading metaheuristic for the MQCP. In `TSQC`, intensification and diversification strategies are applied through an adaptive multi-start mechanism, yielding high-quality solutions across diverse graph instances. Parameter selection, however, remains fixed across all problem instances based on manual tuning. To overcome this limitation, `TSQC` is extended using a Graph Neural Network (GNN)–based learning component, trained to predict algorithm parameters from graph characteristics.

The resulting Deep Learning–guided `TSQC`, referred to as `DeepTSQC`, improves the original algorithm by integrating adaptive parameter selection learned from structural graph features. On dense benchmark instances, `DeepTSQC` matches `TSQC` in maximum quasi-clique size while consistently reducing average running times by factors of 2–4. On sparse real-life networks, `DeepTSQC` consistently achieves equal or higher objective values and demonstrates runtime improvements, particularly at higher density thresholds. These findings demonstrate that data-driven, instance-specific parameter tuning significantly enhances `TSQC`'s performance across diverse MQCP instances, delivering substantial speedups with negligible compromise on solution quality. Nonetheless, the additional overhead to predict parameters and imbalanced class distributions indicate promising directions for future refinement.

The remainder of this thesis is structured as follows: In Section 2, an elaboration on related work is given and Section 3 formally describes the MQCP. Further, the existing `TSQC` algorithm is discussed in Section 4, and the GNN-based parameter selection, resulting in `DeepTSQC`, is elaborated on in Section 5. Then, the exact experiments conducted are detailed in Section 6, and Section 7 provides the results along with a discussion on potential limitations. Lastly, a conclusion is given in Section 8, including suggestions for further research directions.

2 Related Work

The Maximum Quasi-Clique Problem (MQCP), an extension of the Maximum Clique Problem, has gained attention due to its applicability to noisy and incomplete real-world graph data, such as biological or social networks. In this section, exact methods, heuristics, and recent Graph Neural Network (GNN) developments are discussed.

Pattillo et al. (2013) laid foundational work by proving MQCP's computational complexity as $\mathcal{NP}$-hard and presenting initial Mixed Integer Linear Programming (MILP) formulations. Subsequently, Pajouh et al. (2014) proposed a tailored Branch-and-Bound algorithm, enhancing performance relative to initial MILP solvers. Veremyev et al. (2016) further refined these formulations specifically for sparse graphs. Nevertheless, exact algorithms remain challenging for large-scale sparse instances, as demonstrated by Ribeiro & Riveaux (2019), who emphasized the need for scalable heuristics.

Heuristics have successfully addressed MQCP scalability. One influential approach is the Adaptive Multi-Start Tabu Search (`AMTS`) by Wu & Hao (2013) for the Maximum Clique Problem, a restricted MQCP variant. Djeddi et al. (2019) adapted this method into the Tabu Search Quasi-Clique (`TSQC`) algorithm, generalizing its applicability. Although `TSQC` achieved strong results via intensification and diversification strategies, its effectiveness was limited by random tie-breaking during vertex selection. Chen et al. (2021) improved this by introducing a secondary scoring function to reduce randomness, enhancing both solution quality and runtime.

Recent combinatorial optimization approaches incorporate Geometric Deep Learning, particularly GNNs, to enhance exact methods and heuristics. Prominent architectures include Graph Convolutional Networks (GCN) by Kipf & Welling (2017), Graph Attention Networks (GAT) by Veličković et al. (2018), and Graph Isomorphism Networks (GIN) by Xu et al. (2019), each employing neighborhood aggregation over node features to encode structural information. Xu et al. (2019) formally demonstrated GIN's superior ability to differentiate graph structures, motivating its selection in this research.

Applications of GNNs in $\mathcal{NP}$-hard problems are promising. Kool et al. (2019) trained a GAT with reinforcement learning to devise routing heuristics, though scaling to larger, sparser graphs posed challenges. Additionally, Gasse et al. (2019) employed a GCN within an imitation learning framework to *learn* branching policies for exact methods, outperforming classical rules and better generalizing to larger graphs, though generalization across varying densities remains untested. In conclusion, the literature suggests that enhancing `TSQC` through GNN-driven parameter selection holds potential, motivating this study's focus.

3 Problem Description

This section mathematically formalizes the problem. Let $G = (V, E)$ be an undirected graph. For all $S \subseteq V$, the set of edges induced by S is given by $E(S) = \{(i, j) \in E \mid i, j \in S\}$. A γ-quasi-clique is then defined as follows:

Definition 1 (γ-quasi-clique)**.** Given a density threshold $\gamma \in (0, 1]$, a subset $S \subseteq V$ is called a γ-quasi-clique if and only if

$$|E(S)| \geq \gamma \binom{|S|}{2}.$$

Consequently, a k-γ-quasi-clique is a γ-quasi-clique such that $|S| = k$. Using Definition 1, the Maximum Quasi-Clique Problem is mathematically formalized below.

Definition 2 (Maximum Quasi-Clique Problem)**.** Given a density threshold $\gamma \in (0, 1]$, the Maximum Quasi-Clique Problem (MQCP) is given by

$$\max_{S \subseteq V} \left\{ |S| : |E(S)| \geq \gamma \binom{|S|}{2} \right\}.$$

In the next two Sections, the solution methods for the MQCP are presented. Specifically, Section 4 describes the conceptual and mathematical underpinnings of `TSQC`, the Multi-Start Tabu Search algorithm. Further, the selection of algorithm parameters using GNNs, resulting in `DeepTSQC`, is explained in Section 5.

4 Multi-Start Tabu Search

The **T**abu **S**earch **Q**uasi-**C**lique (`TSQC`) algorithm by Djeddi et al. (2019) consists of several components. Each of these components is elaborated upon in this section. First, the global concept and pseudo-code for the algorithm is given in Section 4.1. Next, details on intensification and diversification are covered in Sections 4.2 and 4.3, respectively. Moreover, tabu management is described in Section 4.4 and lastly, the construction of initial solutions and restarts is explained in Section 4.5.

4.1 The `TSQC` Algorithm

`TSQC` leverages the quasi-heredity property by Pattillo et al. (2013), which implies that any γ-quasi-clique contains a smaller γ-quasi-clique. This property underscores the local search approach by Djeddi et al. (2019): starting from an initial subset $S \subseteq V$ such that $|S| = k$, vertex swaps are performed iteratively to increase the density until the threshold required for a k-γ-quasi-clique is reached, after which k is incremented and the search for a larger k-γ-quasi-clique is repeated.

To establish this, a solution space and objective function are required throughout `TSQC`. Let $\Omega_k = \{S \subseteq V : |S| = k\}$ be the solution space for finding a k-γ-quasi-clique. Moreover, let $f(S) = \sum_{\{u,v\} \subseteq S} e_{uv}$ denote the number of edges present in any $S \in \Omega_k$, where e_{uv} is a binary indicator for whether edge (u, v) is present in the graph. Having defined Ω_k and $f(S)$, the

global outline for TSQC is provided below in Algorithm 1. Note that the TSQ sub-algorithm is invoked to execute intensification and diversification moves under tabu constraints. During TSQ, $f(S)$ is maximized until the quasi-clique criterion is reached or no improvement is found for L consecutive iterations, marking L as the search depth. Once a k-γ-quasi-clique is found, k is incremented, and the search continues until the maximum number of iterations is reached, at which point the best solution found is returned.

Algorithm 1 TSQC Algorithm

Require: $G = (V, E)$, γ, max iterations I_{max}
Ensure: k-γ-quasi-clique if found

$k \leftarrow 2$
$i \leftarrow 0$, $S^{\text{best}} \leftarrow \varnothing$
while $i < I_{max}$ **do**
  $S \leftarrow \texttt{initialSolution}(\Omega_k)$
  while $i < I_{max}$ **do**
    $S^* \leftarrow \texttt{TSQ}(G, \gamma, f(\cdot), S, k, i)$
    if $f(S^*) \geq \gamma\binom{k}{2}$ **then**
      $S^{\text{best}} \leftarrow S^*$
      break
    end if
    $S \leftarrow \texttt{newInitialSolution}(\Omega_k)$
  end while
  $k \leftarrow k + 1$
end while
return S^{best}

Algorithm 2 TSQ Algorithm

Require: $G = (V, E)$, γ, S, k, i, depth L
Ensure: Best solution S^*

$S^* \leftarrow S$, $f^* \leftarrow f(S)$, $I \leftarrow 0$
while $I < L$ **do**
  if improvement possible in G **then**
    $S \leftarrow \texttt{intensify}(S)$
  else
    $S \leftarrow \texttt{diversify}(S)$
  end if
  $\texttt{updateTabuLists()}$, $i \leftarrow i + 1$
  if $f(S) \geq \gamma\binom{k}{2}$ **then return** S
  end if
  if $f(S) > f^*$ **then**
    $S^* \leftarrow S$, $f^* \leftarrow f(S)$, $I \leftarrow 0$
  else
    $I \leftarrow I + 1$
  end if
end while
return S^*

4.2 Intensification

One of TSQC's strengths is efficient exploration during intensification, by constraining the solution's neighborhood. In particular, at each iteration of TSQ, it is determined whether an improvement on S is possible in the graph $G = (V, E)$. Let $d_S(v) = |\{u \in S : (u, v) \in E\}|$ be the degree of vertex $v \in V$ with respect to S. Then, $\underline{d}_S = \min_{u \in S \setminus T_{\text{out}}} d_S(u)$ and $\overline{d}_S = \max_{v \in V \setminus \{S \cup T_{\text{in}}\}} d_S(v)$ are computed, representing the minimum degree among non-tabu vertices in S and the maximum degree among non-tabu vertices outside S, respectively. Here, T_{out} and T_{in} denote the sets of vertices marked as 'tabu' for removal from, and addition to the set $S \subseteq V$, respectively. Using $\underline{d}_S$ and $\overline{d}_S$, the critical sets A and B are defined as follows:

$$A = \{u \in S \setminus T_{\text{out}} : d_S(u) = \underline{d}_S\}, \tag{1}$$

$$B = \{v \in V \setminus \{S \cup T_{\text{in}}\} : d_S(v) = \overline{d}_S\}. \tag{2}$$

Then, the set of all possible swaps is $A \times B$. To identify potentially improving moves, the difference in the objective function is computed. Let $S' = S \setminus \{u\} \cup \{v\}$ be the solution after applying swap $(u, v) \in A \times B$. Then, the gain of swap (u, v) is given by

$$\Delta_{uv} = f(S') - f(S) = d_S(v) - d_S(u) - e_{uv}. \tag{3}$$

As mentioned by Djeddi et al. (2019) and Wu & Hao (2013), this gain can be computed conveniently by the righthand side of equation (3). However, the authors do not prove this, therefore a formal proof is provided in Appendix A. By construction, for all $(u, v) \in A \times B$ it holds that $\Delta_{uv} = \overline{d}_S - \underline{d}_S - e_{uv}$. Hence, the best swaps are $T = \{(u, v) \in A \times B : (u, v) \notin E\}$. Using T, intensification is performed according to the following order of steps:

1. If $\overline{d}_S - \underline{d}_S \geq 0$ and $T \neq \emptyset$, then a pair (u, v) is chosen uniformly at random from T.
2. Otherwise, if $\overline{d}_S - \underline{d}_S - 1 \geq 0$, then a pair (u, v) is chosen uniformly at random from $A \times B$.
3. Otherwise, no improving swap exists in $A \times B$, and the diversification procedure (see Section 4.3) is executed.

4.3 Diversification

The move selection strategy ensures an exhaustive constrained neighborhood exploration. However, when no improving swap exists, diversification is applied to escape potential local optima and steer the search into previously unexplored regions. Although solution deterioration may occur, this diversification is triggered in a controlled, probabilistic manner, similar to the approaches in Djeddi et al. (2019) and Wu & Hao (2013).

In particular, let $l = \gamma\binom{k}{2} - f(S)$ denote the feasibility gap of the current solution S. Then, with probability $P = \min\{(l+2)/k,\, 0.1\}$, a worse swap is selected by choosing $(u, v) \in S \times (V \setminus S)$ uniformly at random under the constraint $d_S(v) < h$, where $h = \lfloor 0.85\,\gamma k \rfloor$ if $\rho \leq 0.5$ and $h = \lfloor \gamma k \rfloor$ otherwise, and $\rho = |E|/\binom{|V|}{2}$ denotes the graph density. This choice of h causes greater solution deterioration in sparser graphs, reflecting the increased difficulty of escaping local optima. Conversely, with probability $1 - P$, a random swap from T is selected in case $T \neq \emptyset$. Otherwise, select a swap uniformly at random from $A \times B$. Since P decreases as the feasibility gap l shrinks, the likelihood of deterioration diminishes once the search approaches the density threshold γ. In the next section, tabu list management is explained.

4.4 Tabu Management

The tabu mechanism prevents cycling by forbidding recently swapped vertices from being reversed immediately. Specifically, once a swap $(u, v) \in A \times B$ has been applied, the number of iterations during which these vertices remain 'tabu' is determined. These durations, known as tabu tenures, are adaptively computed by the following equations:

$$T_u = \lceil l \rceil + \text{rand}(0, \max\{0, C - 1\}), \tag{4}$$

$$T_v = \lceil \alpha l \rceil + \text{rand}(0, \max\{0, \lfloor \alpha C \rfloor - 1\}), \tag{5}$$

where $C = \max\{\lfloor \frac{k}{40} \rfloor, 6\}$ and rand(a,b) returns a uniform random integer on [a,b]. The tenure T_u for removed vertices is likely to exceed T_v for added vertices due to $\alpha = 0.6$, reflecting that promising vertices are more likely to be re-added than poor vertices are to be re-removed. The edge deficit l directly influences tenures: larger deficits yield longer tenures, preventing return to less promising regions. The random component adds variability to prevent the search from falling into predictable patterns. The parameter C scales with the target clique size k to ensure larger problem instances have proportionally longer tabu tenures (on average), with a minimum value of 6 to maintain effectiveness even for small instances where $k < 240 = 40 \cdot 6$. Next, the strategies for generating initial and restart solutions are elaborated on.

4.5 Restart Strategy

Solutions are initialized using a greedy constructive heuristic: beginning from a random vertex, vertices are added iteratively to maximize connectivity. Specifically, for a partial solution S with $|S| < k$, a vertex $v \in \arg\max_{u \in V \setminus S}\{d_S(u)\}$ is selected (with ties broken uniformly at random) until $|S| = k$.

If `TSQ` fails to identify a k-γ-quasi-clique within L non-improving iterations, a restart is performed based on long-term frequency memory (Djeddi et al., 2019). Each vertex v is assigned a counter g_v, which is incremented by 1 whenever v participates in an applied swap. The restart solution is constructed by first choosing $v = \arg\min_{u \in V} g_u$ and then iteratively adding vertices that maximize $d_S(v)$, with ties broken first by minimal g_v and then uniformly at random.

To prevent stagnation, the frequency memory is reset whenever any counter exceeds k (i.e. $\exists\, v \in V : g_v > k$), upon which all g_v are set to zero. Since larger quasi-cliques naturally require more iterations, the threshold is dependent on k to maintain the effectiveness of the frequency mechanism across varying graph sizes.

5 Search Depth Policy

This section outlines the methodology for Deep Learning-Guided `TSQC`, referred to as `DeepTSQC`. First, a motivation for the choice of this approach is given. Further, Section 5.1 briefly introduces the overall concept of Graph Neural Networks (GNNs) and the formal prediction task is outlined in Section 5.2. Section 5.3 provides mathematical details on the chosen model architecture and lastly, model training is described in Section 5.4.

Although Djeddi et al. (2019) demonstrate effective performance of `TSQC` across diverse graph instances, `TSQC` heavily depends on the search depth parameter L. The authors select a fixed value of L based on a limited grid search and averaged runtimes from 10 runs per graph to address randomness. However, this selection used inconsistent values of γ and assumed the optimal L is independent of the initial random solution, an assumption potentially invalid for larger, sparser graphs. Thus, in `DeepTSQC` the optimal L is predicted adaptively per graph and starting solution using GNNs, motivated by the explicit observation in Djeddi et al. (2019) that '*L is very sensitive to the structure of the graph and must be adjusted carefully*'. For reference, a visual distinction between `TSQC` and `DeepTSQC` when searching for a γ-quasi-clique of size k is presented in Figure 1.

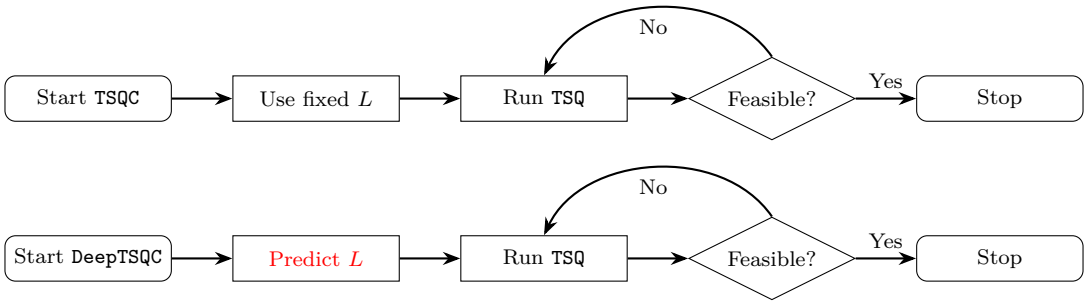


Figure 1: Global algorithm flowcharts for finding a k-γ-quasi-clique.

5.1 Introduction to GNNs

Prior to describing details on the prediction task and the exact model used in this research, this section provides a general introduction to the concept behind GNNs. Conventional Neural Networks (NNs) typically take matrices or vectors as input. Hence, it is not immediately intuitive how to represent graphs in a format compatible with Deep Learning. However, one can represent the graph as a node feature matrix $\mathbf{X} \in \mathbb{R}^{|V| \times f}$, where f is the number of features per node, and utilize the adjacency structure in *message passing* layers. In other words, each vertex learns from their neighbors in a single layer, referred to as an updating step. When stacking multiple message passing layers, vertices learn not only from their neighbors, but also from the neighbors of their neighbors, allowing the information in $\mathbf{X}$ to flow through the graph.

After a certain number of message passing layers, one can aggregate the updated node feature vectors, referred to as embeddings, into a single graph-level embedding. This aggregation is often done using averaging, summing or taking the maximum (Xu et al., 2019). With this graph-level embedding, a traditional NN can be employed for final classification.

5.2 Prediction Task

Given a graph $G = (V, E)$ and $\gamma \in (0, 1]$ it is assumed that the optimal value of L depends on two additional factors: the target quasi-clique size k and the starting solution. Therefore, a GNN is trained to predict L once an initial solution for the search for a k-γ-quasi-clique is constructed in line 4 of Algorithm 1. Mathematically, a policy $\psi_\theta \colon \mathcal{S} \to \mathcal{A}$ parametrized by θ is approximated, where $\mathcal{S}$ and $\mathcal{A}$ denote state and action spaces, respectively. Here, $\mathcal{A}$ is a finite set containing values for L. To ensure a fair comparison, $\mathcal{A}$ equals the grid used in Djeddi et al. (2019). Moreover, the state space elements are given by

$$\mathbf{X} = \begin{bmatrix} \mathbf{x}_1 & \mathbf{x}_2 & \cdots & \mathbf{x}_{|V|} \end{bmatrix}^T, \tag{6}$$

where $\mathbf{X}$ denotes the node feature matrix of a graph $G = (V, E)$. For a given $\mathbf{X} \in \mathcal{S}$, the GNN outputs a probability distribution over the values in $\mathcal{A}$, denoted by $\{p_\theta(L \mid \mathbf{X}) : L \in \mathcal{A}\}$, where θ is the collection of trained model parameters. Using this distribution, the policy $\psi_\theta(\mathbf{X}) = \arg\max_{L \in \mathcal{A}} \{p_\theta(L \mid \mathbf{X})\}$ is deployed in `DeepTSQC`.

To learn geometric patterns over the input graphs and to interact with problem parameters, node features capturing structural information are included in $\mathbf{X}$. Specifically, the clustering coefficient and the core number are incorporated.

Definition 3 (Core Number)**.** Let $G = (V, E)$ be a graph and, for each integer $k \geq 0$, let G_k denote the k-core of G, i.e. the maximal subgraph (in terms of vertices) in which every vertex has degree at least k. Then, the *core number* of a vertex $v \in V$ is $\kappa(v) = \max\{\, k : v \in G_k\}$.

Definition 4 (Clustering Coefficient)**.** Let $G = (V, E)$ be an undirected graph and let $d(v)$ be the degree of vertex $v \in V$. Denote by $N(v) = \{\, u \in V : (u, v) \in E\}$ the neighborhood of v, and let $E(N(v)) = \{\, (u, w) \in E : u, w \in N(v)\}$ be the set of edges induced by $N(v)$. Then, the *clustering coefficient* of v is defined by

$$C(v) = \begin{cases} \frac{\left|E(N(v))\right|}{\binom{d(v)}{2}} & \text{if } d(v) \geq 2 \\ 0 & \text{otherwise} \end{cases}$$

Besides the core numbers and clustering coefficients, nodes are further encoded using the exhaustive list of node features in Table 1. For simplicity, graph-level features are incorporated into $\mathbf{x}_v$ rather than employing more advanced architectures that account for these separately.

Table 1: Elements of $\mathbf{x}_v$.

level	symbol	description	complexity (per vertex)
vertex	$d(v)$	the degree of v	$\mathcal{O}(1)$
	$d_S(v)$	the degree of v with respect to $S \subseteq V$	$\mathcal{O}(d(v))$
	$\kappa(v)$	the core number of v (Definition 3)	$\mathcal{O}(\lvert E\rvert)$
	$C(v)$	the clustering coefficient of v (Definition 4)	$\mathcal{O}(d(v)^2)$
	$\text{dist}_S(v)$	the shortest path from v to any $u \in S$	$\mathcal{O}(\lvert V\rvert + \lvert E\rvert)$
graph	ρ	the density of the graph	
	k	the target quasi-clique size	
	γ	the target clique-density threshold	

5.3 Model Architecture

Engaging with the formally defined prediction task, this section describes the exact model architecture used to approximate ψ_θ. The overall architecture comprises a stack of K Graph Isomorphism Network (GIN) layers, an aggregation layer, and a final Multi-Layer Perceptron (MLP) for graph-level classification.

Figure 2 demonstrates a visual overview of the architecture. Note, until the aggregation layer is reached, each layer receives node embeddings, denoted by $\mathbf{h}_v^{(k)}$ for layer k, from its preceding layer. However, the aggregation layer aggregates these into 1 graph-level embedding of the same dimension, denoted by $\mathbf{h}_G$, and parses this through an MLP to produce the probability distribution $\{p_\theta(L \mid \mathbf{X}) : L \in \mathcal{A}\}$. The mathematical underpinnings of graph isomorphism and the GIN layer are given below, followed by more mathematical details on the aggregation layer and the final MLP.

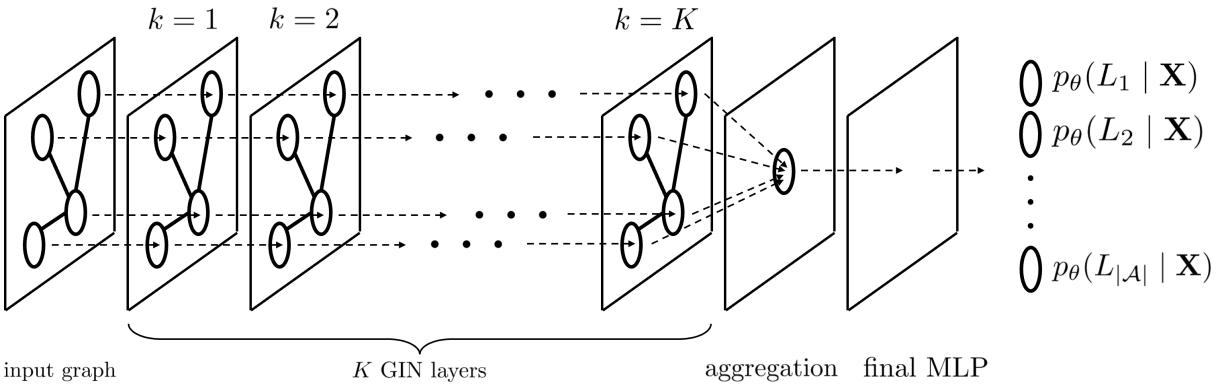


Figure 2: Model architecture.

Recall that graph isomorphism is the condition where two graphs are identical by relabeling vertices via a bijection that preserves all edges. Formally, two graphs $G = (V_G, E_G)$ and $H = (V_H, E_H)$ are *isomorphic* if there exists a bijection $f\colon V_G \longrightarrow V_H$, such that for every $u, v \in V_G$,

$$\{u, v\} \in E_G \iff \{f(u), f(v)\} \in E_H.$$

Then, in the context of GNNs, *expressiveness* denotes their ability to distinguish non-isomorphic graphs. Therefore, the Graph Isomorphism Network is employed as the message-passing backbone, as Xu et al. (2019) have proven it to be the most *expressive* architecture. Mathematically, for every node v at GIN layer $k \in \{1, \ldots, K\}$, the embedding $\mathbf{h}_v^{(k)}$ is updated by first computing

$$\tilde{\mathbf{h}}_v^k = \left(1 + \epsilon^{(k)}\right) \mathbf{h}_v^{(k)} + \sum_{u \in N(v)} \mathbf{h}_u^{(k)},$$

which is the message passing mechanism using the adjacency structure of the graph. Note that $N(v)$ is the neighborhood of vertex v. Further, $\mathbf{h}_v^{(0)} = \mathbf{x}_v$, and $\epsilon^{(k)}$ is a learnable model parameter. Secondly, the embedding is updated as:

$$\mathbf{h}_v^{(k+1)} = \mathrm{MLP}^{(k)}\left(\tilde{\mathbf{h}}_v^{(k)}\right),$$

where, $\mathrm{MLP}^{(k)}(\cdot)$ is the output of an MLP for GIN layer k. Concretely, let L_k be the number of layers in $\mathrm{MLP}^{(k)}$, and let $\mathrm{ReLU}(x) = \max\{0, x\}$ be the activation function used by the MLP to approximate complex, non-linear relationships. Setting $\mathbf{z}_v^{(k,0)} = \tilde{\mathbf{h}}_v^{(k)}$, the MLP computes for each $\ell = 1, \ldots, L_k$,

$$\mathbf{z}_v^{(k,\ell)} = \mathrm{ReLU}\Big(\mathbf{W}^{(k,\ell)}\, \mathbf{z}_v^{(k,\ell-1)} + \mathbf{b}^{(k,\ell)}\Big),$$

and then sets $\mathrm{MLP}^{(k)}\big(\tilde{\mathbf{h}}_v^{(k)}\big) = \mathbf{z}_v^{(k,L_k)}$, where each $\mathbf{W}^{(k,\ell)} \in \mathbb{R}^{d \times d}$ and $\mathbf{b}^{(k,\ell)} \in \mathbb{R}^d$ are learnable parameters. Here, d is the number of neurons in each layer, which is left as a hyperparameter along with the value for L_k. Note, as $\mathbf{h}_v^{(0)} = \mathbf{x}_v$, the first layer in $\mathrm{MLP}^{(1)}$ has a dimension equal to the number of columns in $\mathbf{X}$. Figure 3 provides a visualization of the updating step between two consecutive GIN layers for a single node v, which is annotated with blue. In this example, $L_k = 3$ and $d = 4$.

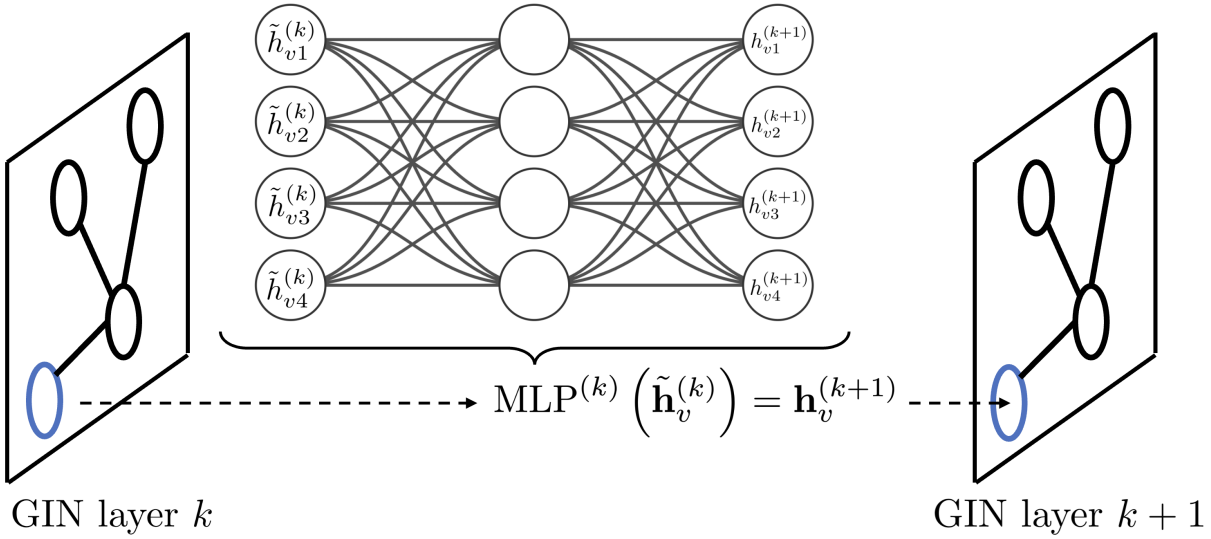


Figure 3: Updating step between consecutive GIN layers.

To collapse the collection of node embeddings into a single graph-level vector, a permutation-invariant readout function is employed in the aggregation layer. A function $f\colon \{\mathbf{h}_v : v \in V\} \longrightarrow \mathbb{R}^d$ is called *permutation-invariant* if, for any permutation π of the vertex indices,

$$f(\{\mathbf{h}_v : v \in V\}) = f(\{\mathbf{h}_{\pi(v)} : v \in V\}).$$

This property is essential since a graph has no inherent ordering on its vertices. Therefore, the readout must produce the same $\mathbf{h}_G$ regardless of how the vertices are ordered. Concretely, the graph-level embedding is computed as follows:

$$\mathbf{h}_G = \mathrm{AGG}\left(\left\{\mathbf{h}_v^{(K)} : v \in V\right\}\right).$$

The exact $\mathrm{AGG}(\cdot)$ function is left as a hyperparameter, and chosen to be one of

$$\sum_{v\in V} \mathbf{h}_v^{(K)}, \quad \frac{1}{|V|}\sum_{v\in V} \mathbf{h}_v^{(K)}, \quad \max_{v\in V}\left\{\mathbf{h}_v^{(K)}\right\}.$$

Each of these operators is permutation-invariant by construction, guaranteeing that isomorphic graphs, differing only by a relabeling of vertices, map to the same graph embedding $\mathbf{h}_G \in \mathbb{R}^d$.

Lastly, the aggregated embedding $\mathbf{h}_G \in \mathbb{R}^d$ is passed through an MLP whose last layer has one output neuron per $L \in \mathcal{A}$ to output a graph-level prediction. Denote by M the total number of layers in this MLP. Let

$$\mathbf{z}^{(0)} = \mathbf{h}_G, \qquad \mathbf{z}^{(\ell)} = \mathrm{ReLU}(\mathbf{W}_f^{(\ell)}\,\mathbf{z}^{(\ell-1)} + \mathbf{b}_f^{(\ell)}), \quad \ell = 1, \ldots, M-1,$$

where for each hidden layer ℓ the weights $\mathbf{W}_f^{(\ell)} \in \mathbb{R}^{d\times d}$ and biases $\mathbf{b}_f^{(\ell)} \in \mathbb{R}^d$ are learnable parameters. The final output layer is then

$$\mathbf{z}^{(M)} = \mathbf{W}_f^{(M)}\,\mathbf{z}^{(M-1)} + \mathbf{b}_f^{(M)},$$

with $\mathbf{W}_f^{(M)} \in \mathbb{R}^{|\mathcal{A}|\times d}$ and $\mathbf{b}_f^{(M)} \in \mathbb{R}^{|\mathcal{A}|}$, so that $\mathbf{z}^{(M)} \in \mathbb{R}^{|\mathcal{A}|}$. Finally, the softmax function is applied over the elements in $\mathbf{z}^{(M)}$ to obtain the probability distribution

$$\left\{ p_\theta(L \mid \mathbf{X}) = \frac{\exp(u_L)}{\sum_{L'\in\mathcal{A}} \exp(u_{L'})} : L \in \mathcal{A} \right\}.$$

5.4 Training

To train the model and estimate the parameters θ in the policy $\psi_\theta : \mathcal{S} \to \mathcal{A}$, a representative set of training data, denoted by $\mathcal{D} = \{(\mathbf{X}_i, \tilde{L}_i) \colon i \in \mathbb{N}_N\}$ is obtained, where $\mathbf{X}_i$ and $\tilde{L}_i$ denote the feature matrix and optimal L for observation i, respectively. Given $\mathcal{D}$, the collection of parameters θ is estimated by minimizing the weighted cross-entropy loss function, a common choice in supervised classification tasks. Weights are applied to account for potential class imbalance. Mathematically, the following is computed:

$$\theta^* = \arg\min_\theta \left\{ -\frac{1}{N} \sum_{i=1}^{N} \sum_{L\in\mathcal{A}} w_L \cdot \mathbb{1}\{\tilde{L}_i = L\} \cdot \log(p_\theta(L \mid \mathbf{X}_i)) \right\}, \tag{7}$$

where the weights are given by $w_L = \frac{N}{|\mathcal{A}| n_L}$, with n_L being the number of optimal occurrences for value L in $\mathcal{D}$. This ensures that rare classes (small n_L) are heavily penalized in case $p_\theta(L \mid \mathbf{X}_i)$ is much smaller than 1, where $p_\theta(L \mid \mathbf{X}_i)$ resembles the probability that L is optimal for state $\mathbf{X}_i$, outputted by the final layer.

As the objective function in Equation 7 is non-convex, the Adam optimizer (Kingma & Ba, 2015), a state-of-the-art stochastic gradient descent algorithm, is employed. It combines the benefits of adaptive gradient-based methods by maintaining parameter estimates of both the mean and the variance of the gradients. By adapting the learning rate based on these moment estimates, the algorithm naturally converges faster than fixed learning rate algorithms.

Lastly, to optimize hyperparameters, the dataset $\mathcal{D}$ is split randomly into 80% training data, 10% validation data, and 10% test data. For each candidate hyperparameter configuration, the model is trained on the training split and evaluated on the validation split using the F_1 score. The F_1 score, defined as

$$\mathrm{F}_1 = 2 \cdot \frac{\text{precision} \times \text{recall}}{\text{precision} + \text{recall}},$$

is the harmonic mean of precision and recall and is particularly well suited to imbalanced classification problems, since it balances false predictions across all classes. To select each new trial efficiently, Bayesian optimization (Bergstra et al., 2011) is utilized, which explores the hyperparameter space probabilistically by maximizing expected improvement in F_1. After a fixed number of trials, the configuration with the highest validation F_1 score is chosen to retrain the model on the combined training and validation data. This final model is then evaluated on the test split to assess its out-of-sample performance. For reproducibility, an overview of all hyperparameters and optimal values can be found in Appendix B.

6 Experiments

In this section, the experimental setup is presented to ensure reproducibility. First, the graphs used for algorithm comparison along with execution strategies are described in Section 6.1. Further, the collection of training data for the GNN is elaborated on in Section 6.2. The GNN training and all experiments are executed on a Macbook Pro with an M4 Chip and 16GB of Random Access Memory (RAM). In addition, the solution methods are implemented in Python and all code is publicly available at `github.com/TumGommans/gnn-quasi-clique`, where the instances used for gathering training data can be found as well[1].

6.1 Algorithm Comparison

The performance of `DeepTSQC` is compared with that of `TSQC` by running both algorithms on 15 DIMACS instances, which are rather dense, and 10 real-life sparse graphs originally used by Djeddi et al. (2019). These datasets are publicly available in the Network Repository (Rossi & Ahmed, 2015). For reference, DIMACS graphs are benchmark datasets created by the Centre for **Di**screte **Ma**thematics and Theoretical **C**omputer **S**cience for comparing the performance of graph algorithms.

To maintain consistency with Djeddi et al. (2019), γ is set to $\{0.85, 0.95, 1.0\}$ for the DIMACS instances and to $\{0.5, 0.6, 0.7, 0.8, 0.9\}$ for the real-life graphs. The same initialization approach is used, by initializing the algorithms with $k = \tilde{\omega}_\gamma$, which equals the best known quasi-clique size for each instance-γ pair[2]. Moreover, k is incremented each time a feasible k-γ-quasi-clique is found. Once either an iteration limit of $I_{max} = 10^6$ or a time limit of 10 minutes is reached, the time taken to find the largest γ-quasi-clique is recorded.

Moreover, to account for algorithmic stochasticity, each instance–γ pair is independently executed 10 times using a reproducible seeding scheme: using seed 42, 10 random integers are generated and used as seeds for the independent runs. In case no feasible γ-quasi-clique is found due to initialization at $k = \tilde{\omega}_\gamma$, an objective of 0 is recorded. From these runs, both the maximum and average clique sizes found are recorded, as well as the average running time.

6.2 Training Data

Further, for training the GNN, data is collected by running `TSQC` on a wide variety of real-world graphs for all $\gamma \in \{0.5, 0.6, 0.7, 0.8, 0.9\}$, ensuring the training data is representative. The instances include both biological and ecological graphs from the Network Repository. Moreover, each instance–γ pair is executed five times to produce different initial solutions due to randomness. Each distinct run is carried out for all $L \in \{500, 1000, 5000\}$, and the optimal L is chosen as the value yielding the highest objective, with ties resolved first by shorter runtime and then by smaller L values in accordance with their ordinal nature.

[1] These are available in the `data/training/biological` and `data/training/ecological` directories.
[2] These are available in Tables 1-3 and Tables 6-10 of the research paper by Djeddi et al. (2019).

7 Numerical Results

Having described the experimental setup in Section 6, this section provides the numeric results. First, a comparison between our implementation of `TSQC` and that of Djeddi et al. (2019) is given in Section 7.1. Next, the results for the DIMACS graphs and real-life graphs are discussed in Sections 7.2 and 7.3, respectively. Lastly, Section 7.4 briefly elaborates on potential limitations of `DeepTSQC`.

7.1 Comparison to Djeddi et al. (2019)

This section briefly discusses the reproducibility of `TSQC`, by comparing the results of our implementation to that of Djeddi et al. (2019), across the 15 DIMACS graphs and 10 real-life graphs. Table 2 demonstrates all instance-γ pairs, for which the best objective value found in the 10 independent runs differs across both implementations. The fifth and sixth columns present these maximum objective values for Djeddi et al. (2019) and our implementation of `TSQC`, respectively.

Table 2: Differing results between Djeddi et al. (2019) and ours

instance				`TSQC`	
name	$\|V\|$	ρ	γ	Djeddi	ours
california	6175	0.001	0.6	14	16
harvard500	500	0.016	0.5	37	TL
brock400-2	400	0.749	1.0	29	TL
keller5	776	0.752	0.85	286	290
keller5	776	0.752	0.95	47	48

When inspecting Table 2, differing outcomes occur for five instance-γ pairs. In three cases, our implementation finds larger quasi-cliques, while in two it falls short of the best-known value (denoted TL: Time Limit). These deviations stem from the stochastic nature of `TSQC`, which relies on randomized initial solutions, probabilistic diversification moves, random tie-breaking in vertex selection and adaptive, randomly adjusted tabu tenures. However, such sensitivity appears in only about 5% of all instance-γ pairs used, underscoring `TSQC`'s robustness against its stochastic components.

7.2 Dimacs Results

The numerical results for all DIMACS instances are presented in this section. In Table 3, the fifth and sixth columns demonstrate the maximum and average objective values (ω_γ) for each instance-γ pair, found by both `TSQC` and `DeepTSQC`. If no average is provided, the average is equal to the maximum. Additionally, the seventh and eighth columns show the average running times (t) for both methods. The bolded values indicate either the best objective values, the best running times, or both. Lastly, graphs for which the results do not reveal any insights are shown in Appendix C.

Table 3: Numerical results for DIMACS instances.

instance				ω_γ: max(avg)		t: avg	
name	$\|V\|$	ρ	γ	TSQC	DeepTSQC	TSQC	DeepTSQC
brock200-2	200	0.496	0.85	**19**	**19**	2.1	**0.7**
			0.95	**13**	**13**	6.4	**2.6**
			1.0	12(8.4)	**12(9.6)**	262.8	172.5
brock200-4	200	0.658	0.85	**39**	**39**	2.0	**0.8**
			0.95	**21**	**21**	1.4	**0.8**
			1.0	**17(10.2)**	17(6.8)	293.4	403.0
brock400-2	400	0.749	0.85	**100**	**100**	55.0	**41.2**
			0.95	**40**	**40**	**71.7**	88.2
			1.0	TL	TL	TL	TL
brock400-4	400	0.749	0.85	102(81.6)	**102(91.8)**	373.1	202.0
			0.95	**39**	**39**	34.9	**14.9**
			1.0	33(3.3)	**33(13.2)**	598.6	536.1
keller4	171	0.649	0.85	**31**	**31**	**1.1**	1.5
			0.95	**15**	**15**	1.1	**0.5**
			1.0	**11**	**11**	**0.1**	**0.1**
keller5	776	0.752	0.85	**290(172.3)**	288(258.6)	444.1	360.2
			0.95	**48(47.2)**	47	128.0	58.0
			1.0	**27(21.6)**	27(2.7)	311.1	540.5
p-hat1500-2	1500	0.506	0.85	**487**	**487**	**26.2**	26.6
			0.95	**193**	**193**	**6.0**	6.3
			1.0	**65**	**65**	**8.3**	11.1
p-hat1500-3	1500	0.754	0.85	**943**	**943**	**74.0**	75.1
			0.95	**351**	**351**	**16.0**	16.7
			1.0	**94**	**94**	74.3	**74.2**
p-hat300-1	300	0.244	0.85	**12**	**12**	1.0	**0.4**
			0.95	**9**	**9**	2.2	**1.2**
			1.0	**8**	**8**	0.9	**0.3**
p-hat300-3	300	0.744	0.85	**180**	**180**	**0.5**	0.6
			0.95	**71**	**71**	**0.2**	**0.2**
			1.0	**36**	**36**	3.6	**1.0**
p-hat700-1	700	0.249	0.85	**19**	**19**	12.3	**3.1**
			0.95	**13**	**13**	14.1	**7.4**
			1.0	**11**	**11**	17.9	**13.3**
p-hat700-2	700	0.498	0.85	**223**	**223**	**2.6**	2.7
			0.95	**96**	**96**	0.9	**0.7**
			1.0	**44**	**44**	1.8	**1.4**
p-hat700-3	700	0.748	0.85	**430**	**430**	**6.9**	7.0
			0.95	**176**	**176**	2.5	**2.4**
			1.0	**62**	**62**	5.1	**3.5**

Table 3 shows the results for the DIMACS instances. First, out of the seven instance-γ pairs where `TSQC` and `DeepTSQC` do not consistently find the same objective value, `TSQC` finds a larger average objective value in four cases. This indicates `TSQC` is slightly more consistent in finding high-quality solutions, though the maximum values remain equal in most cases. However, regarding computational efficiency, `DeepTSQC` often outperforms `TSQC` in terms of average running times, mainly by factors of 2-4. This is particularly evident in instances-γ pairs like (p-hat300-3, $\gamma = 1.0$) and (p-hat700-1, $\gamma = 0.85$), where `DeepTSQC` on average converges approximately 4 times faster.

These findings suggest that for rather dense graphs, the comparison between `TSQC` and `DeepTSQC` resembles the classic tradeoff between solution quality and computation time. However, on dense DIMACS graphs, `DeepTSQC` proves to be the more suitable algorithm, as it achieves superior running times more consistently, whereas `TSQC` only surpasses it in solution quality by 1 count.

7.3 Real-Life Results

Having discussed the results on dense graphs, the outcomes on sparse real-life networks are discussed in this section. In Table 4 on the next page, the fifth and sixth columns demonstrate the maximum and average objective values (ω_γ) for each instance-γ pair, found by both `TSQC` and `DeepTSQC`. If no average is provided, the average is equal to the maximum. Additionally, the seventh and eighth columns show the average running times (t) for both methods. The bolded values indicate either the best objective values, the best running times, or both. Similarly to the DIMACS instances, real-life graphs for which the results do not reveal any insights are shown in Appendix C.

Table 4 reveals patterns that both complement and contrast with the DIMACS findings. First, `DeepTSQC` maintains its computational superiority across most real-world instances, though the margins vary significantly depending on γ and the graph structure. On sparse networks like california and geom ($\rho = 0.001$), `DeepTSQC` consistently outperforms `TSQC` in runtime, with particularly notable differences for higher γ values. The algorithm additionally demonstrates superior consistency in finding the highest objective value, as seen in california where `DeepTSQC` achieves higher average objective values across multiple density thresholds. Other examples of this are ca-grqc for $\gamma = 0.5$, and netscience for $\gamma = 0.5$. Moreover, two instances reveal scenarios where `TSQC` outperforms `DeepTSQC` in runtime, such as netscience with $\gamma = 0.6$ (116.8s vs 148s) and harvard500 with $\gamma = 0.6$ (6.3s vs 7.4s). This suggests that the running time improvement from `DeepTSQC` may be graph-dependent, possibly due to how γ relates to the specific structural properties of the instances.

In conclusion, the generally consistent solution quality across both groups of graphs, paired with frequent runtime improvements, makes `DeepTSQC` a more suitable algorithm for the MQCP, though the occasional performance variations suggest that adaptive search depth selection is not exclusively beneficial for diverse real-world problem instances. Hence, although the result is promising, `DeepTSQC` still carries along certain limitations, which are discussed in Section 7.4.

Table 4: Numerical results for real life instances.

instance				ω_γ: max(avg)		t: avg	
name	$\|V\|$	ρ	γ	TSQC	DeepTSQC	TSQC	DeepTSQC
california	6175	0.001	0.5	**26**	**26**	29.6	**23.4**
			0.6	16(14.8)	**16(15.2)**	330.7	391.9
			0.7	14(5.6)	**14(9.8)**	452.0	309.3
			0.8	13(5.2)	**13(7.8)**	418.7	352.5
			0.9	12(4.8)	**12(7.2)**	524.8	379.2
harvard500	500	0.016	0.5	TL	TL	TL	TL
			0.6	**29**	**29**	**6.3**	7.4
			0.7	**26**	**26**	10.3	**7.8**
			0.8	**24**	**24**	10.9	**4.3**
			0.9	**23**	**23**	13.7	**5.7**
ca-grqc	4158	0.002	0.5	81(24.3)	**81(40.5)**	485.5	446.8
			0.6	**63**	**63**	9.8	**9.7**
			0.7	**57**	**57**	10.3	**8.5**
			0.8	**52**	**52**	**7.7**	**7.7**
			0.9	**49**	**49**	8.1	**7.2**
email	1133	0.009	0.5	**25**	**25**	12.4	**9.1**
			0.6	**20**	**20**	8.7	**6.5**
			0.7	**17**	**17**	8.3	**5.9**
			0.8	**15**	**15**	20.2	**12.5**
			0.9	**13**	**13**	12.5	**11.0**
geom	6158	0.001	0.5	**44**	**44**	1.9	**1.7**
			0.6	**36**	**36**	2.5	**1.8**
			0.7	**30**	**30**	20.0	**17.1**
			0.8	**26**	**26**	18.9	**16.8**
			0.9	**23**	**23**	51.2	**24.2**
netscience	1461	0.003	0.5	31(21.7)	**31**	301.1	166.5
			0.6	**27**	**27**	**116.8**	148.0
			0.7	**24**	**24**	33.5	**13.4**
			0.8	**22**	**22**	26.8	**13.2**
			0.9	**21**	**21**	34.2	**29.2**

7.4 Limitations

This section elaborates both on the out-of-sample performance of the GNN in DeepTSQC, and its practical limitations. First, the gathering of training data shows a surprising result. The optimal value L never equals 5000, meaning the 3-class prediction task scales down to binary classification. In addition, the resulting distribution between $L = 500$ and $L = 1000$ is highly skewed towards $L = 500$, with 370 occurrences compared to 5. This further underscores the importance of the weights in Equation (7), to heavily penalize false predictions for $L = 1000$. After training the model on 90% of data using the optimal hyperparameter configuration, the remaining 10% holdout is used to perform a final evaluation by computing both the accuracy (percentage of correct predictions) and the F_1 score.

The resulting accuracy equals 94.7%, with an F_1 score of 73.6%. The value of the F_1 score compared to the accuracy indicates the GNN highly favors the majority class, being $L = 500$. This comes at no surprise considering the class distribution is imbalanced, highlighting the first limitation of `DeepTSQC`, as a more extensive collection of training data is required to increase out-of-sample performance, possibly across more values of L besides $\{500, 1000, 5000\}$. Additionally, the out-of-sample F_1 score can potentially be increased by oversampling the minority class during training.

Secondly, the GNN currently depends on node features which can be inefficient to compute for large and dense graphs, as demonstrated by their theoretical complexities in Table 1. For instance, recall the clustering coefficient $C(v)$, which runs in $\mathcal{O}(d(v)^2)$. In dense graphs it can hold that $d(v) = \mathcal{O}(|V|)$ and computing $C(v)$ for each is then $|V| \cdot \mathcal{O}(|V|^2) = \mathcal{O}(|V|^3)$. This is rather costly for large dense graphs. Although in practice many real-world graphs are extremely sparse, an interesting research direction is to analyze the importance of $C(v)$ in the GNNs predictions, and assess whether this feature can be omitted.

8 Conclusion

This thesis addressed the Maximum Quasi-Clique Problem (MQCP), an $\mathcal{NP}$-hard optimization problem relevant in numerous real-world contexts, such as protein interaction networks, social interactions, and financial systems. Specifically, an approach named Deep Learning–guided Tabu Search for Quasi-Cliques (`DeepTSQC`) was introduced, enhancing the existing `TSQC` algorithm by Djeddi et al. (2019) through adaptive parameter selection using Graph Neural Networks (GNNs).

The developed `DeepTSQC` algorithm successfully leverages structural graph features and algorithm specific states to predict the optimal search depth parameter, significantly improving computational efficiency. Extensive experimentation on both dense benchmark DIMACS instances and sparse real-life network data confirmed that `DeepTSQC` consistently matches or exceeds the performance of the original `TSQC` method. On dense instances, it maintained solution quality while occasionally achieving substantial speedups of approximately 2 to 4 times. For sparse real-life networks, `DeepTSQC` demonstrated increased stability in solution quality, particularly at higher density thresholds, accompanied by consistent improvements in runtime performance.

These findings underscore the practical value of integrating Deep Learning techniques with meta-heuristic algorithms. The benefit of adaptive, data-driven approaches, such as `DeepTSQC`, demonstrate the potential for broader applicability in similar combinatorial optimization problems. While `DeepTSQC` offers notable advantages, further refinements remain essential to manage prediction overhead and improve out-of-sample performance in the presence of class imbalance. Hence, an interesting future research direction is to explore feature importance in the GNN's predictions, potentially allowing the elimination of features that are costly to compute. Moreover, a more extensive collection of training data can smooth out the class imbalance, resulting in a more generalizing model and potentially better algorithmic performance in `DeepTSQC`.

References

Bergstra, J., Bardenet, R., Bengio, Y. & Kégl, B. (2011). Algorithms for hyper-parameter optimization. In *Advances in neural information processing systems 24* (pp. 2546–2554).

Chen, J., Cai, S., Pan, S., Wang, Y., Lin, Q., Zhao, M. & Yin, M. (2021). Nuqclq: An effective local search algorithm for maximum quasi-clique problem. In *Proceedings of the aaai conference on artificial intelligence* (Vol. 35, pp. 12258–12266).

Djeddi, Y., Ait Haddadene, H. & Belacel, N. (2019). An extension of adaptive multi-start tabu search for the maximum quasi-clique problem. *Computers & Industrial Engineering*, *132*, 280–292.

Gasse, M., Chételat, D., Ferroni, N., Charlin, L. & Lodi, A. (2019). Exact combinatorial optimization with graph convolutional neural networks. *Advances in neural information processing systems*, *32*.

Kingma, D. P. & Ba, J. (2015). Adam: A Method for Stochastic Optimization. In *Proceedings of the 3rd international conference on learning representations (iclr).* Retrieved from `https://arxiv.org/abs/1412.6980` (arXiv:1412.6980)

Kipf, T. N. & Welling, M. (2017). Semi-supervised classification with graph convolutional networks. In *International conference on learning representations (iclr).*

Kool, W., van Hoof, H. & Welling, M. (2019). Attention, learn to solve routing problems! In *International conference on learning representations (iclr).*

Pajouh, F. M., Miao, Z. & Balasundaram, B. (2014). A branch-and-bound approach for maximum quasi-cliques. *Annals of Operations Research*, *216*, 145–161.

Pattillo, J., Veremyev, A., Butenko, S. & Boginski, V. (2013). On the maximum quasi-clique problem. *Discrete Applied Mathematics*, *161*(1-2), 244–257.

Ribeiro, C. C. & Riveaux, J. A. (2019). An exact algorithm for the maximum quasi-clique problem. *International Transactions in Operational Research*, *26*(6), 2199–2229.

Rossi, R. A. & Ahmed, N. K. (2015). The network data repository with interactive graph analytics and visualization. In *Aaai.* Retrieved from `https://networkrepository.com`

Veličković, P., Cucurull, G., Casanova, A., Romero, A., Liò, P. & Bengio, Y. (2018). Graph attention networks. In *International conference on learning representations (iclr).*

Veremyev, A., Prokopyev, O. A., Butenko, S. & Pasiliao, E. L. (2016). Exact mip-based approaches for finding maximum quasi-cliques and dense subgraphs. *Computational Optimization and Applications*, *64*(1), 177–214.

Wu, Q. & Hao, J.-K. (2013). An adaptive multistart tabu search approach to solve the maximum clique problem. *Journal of Combinatorial Optimization*, *26*, 86–108.

Xu, K., Hu, W., Leskovec, J. & Jegelka, S. (2019). How powerful are graph neural networks? In *International conference on learning representations (iclr).*

A Correctness of Equation (3)

Proof. Recall that by definition,

$$f(S) = \sum_{\{i,j\} \subseteq S} e_{ij} = \tfrac{1}{2} \sum_{i,j \in S} e_{ij} = \tfrac{1}{2} \sum_{i \in S} d_S(i).$$

Set $S' = S \setminus \{u\} \cup \{v\}$. Then, the following two equations hold:

$$\sum_{i \in S'} d_{S'}(i) = \sum_{i \in S \setminus \{u\}} d_{S'}(i) + d_{S'}(v) = \sum_{i \in S \setminus \{u\}} (d_S(i) - e_{iu} + e_{iv}) + (d_S(v) - e_{uv}),$$
$$\sum_{i \in S} d_S(i) = \sum_{i \in S \setminus \{u\}} d_S(i) + d_S(u).$$

Subtracting gives,

$$\begin{aligned} 2(f(S') - f(S)) &= \sum_{i \in S'} d_{S'}(i) \; - \sum_{i \in S} d_S(i) \\ &= \Big[\sum_{i \in S \setminus \{u\}} (d_S(i) - e_{iu} + e_{iv}) + (d_S(v) - e_{uv}) \Big] - \Big[\sum_{i \in S \setminus \{u\}} d_S(i) + d_S(u) \Big]. \end{aligned}$$

Since $\sum_{i \in S \setminus \{u\}} e_{iu} = d_S(u)$ and $\sum_{i \in S \setminus \{u\}} e_{iv} = d_S(v) - e_{uv}$, it follows that

$$2\Delta_{uv} = 2(f(S') - f(S)) = 2(d_S(v) - d_S(u) - e_{uv})$$ □

B Hyperparameters

The search spaces and optimal values for the GNN hyperparameters are presented in Table 5. The `batch size` equals the number of observations for which the gradient is computed during each iteration of Adam. Further, the `dropout` rate is the fraction of neurons that are deactivated (set to 0) in each layer during training. This prevents the GNN from relying on specific features and therefore mitigates overfitting. Lastly, the `learning rate` is the baseline gradient descent step size in Adam and `epochs` equals the number of times the optimizer iterates over all training batches.

Table 5: GNN hyperparameter grid and optimal values.

parameter	search space	optimal value
`batch size`	$\{32, 64, 128\}$	32
`dropout`	$[0.0, 0.5]$	0.4998
`epochs`	$\{50, 100, 150\}$	50
`final mlp layers` (L_f)	$\{2, 3\}$	2
`hidden dim` (d)	$\{32, 64, 128\}$	32
`learning rate`	$[10^{-9.21}, 10^{-4.61}]$	$1.24 \cdot 10^{-3}$
`mlp layers per gin` (L_k)	$\{2, 3\}$	2
`num gin layers` (K)	$\{2, 3, 4, 5\}$	5
`readout`	$\{\text{sum}, \text{mean}\}$	mean

C Additional Results

Table 6: Numerical results for DIMACS instances continued.

instance				ω_γ: max(avg)		t: avg	
name	$\|V\|$	ρ	γ	TSQC	DeepTSQC	TSQC	DeepTSQC
hamming8-4	256	0.639	0.85	**35**	**35**	**0.0**	**0.0**
			0.95	**17**	**17**	**0.1**	**0.1**
			1.0	**16**	**16**	**0.1**	**0.1**
p-hat300-2	300	0.489	0.85	**85**	**85**	**0.2**	**0.2**
			0.95	**41**	**41**	0.5	**0.4**
			1.0	**25**	**25**	**0.2**	**0.2**

Table 7: Numerical results for real life instances continued.

instance				ω_γ: max(avg)		t: avg	
name	$\|V\|$	ρ	γ	TSQC	DeepTSQC	TSQC	DeepTSQC
smallw	233	0.037	0.5	**28**	**28**	**0.2**	**0.2**
			0.6	**22**	**22**	0.2	**0.1**
			0.7	**17**	**17**	0.2	**0.1**
			0.8	**14**	**14**	0.2	**0.1**
			0.9	**11**	**11**	0.6	**0.2**
usair97	332	0.039	0.5	**67**	**67**	**0.1**	**0.1**
			0.6	**57**	**57**	**0.1**	**0.1**
			0.7	**49**	**49**	**0.1**	**0.1**
			0.8	**43**	**43**	**0.1**	**0.1**
			0.9	**35**	**35**	**0.0**	**0.0**
as-735	6474	0.001	0.5	**36**	**36**	**0.8**	**0.8**
			0.6	**29**	**29**	**0.5**	0.8
			0.7	**24**	**24**	**0.4**	**0.4**
			0.8	**19**	**19**	**0.3**	**0.3**
			0.9	**15**	**15**	1.3	**0.8**
celegans metabolic	453	0.020	0.5	**30**	**30**	**0.0**	**0.0**
			0.6	**24**	**24**	**0.0**	**0.0**
			0.7	**18**	**18**	0.1	**0.0**
			0.8	**15**	**15**	0.3	**0.2**
			0.9	**12**	**12**	0.2	**0.1**

D Code Execution

All code is publicly available at `github.com/TumGommans/gnn-quasi-clique/`. All classes and methods contain docstrings, including descriptions of input arguments and return statements. To reproduce all the results in this research, run the following command in the terminal:

```
caffeinate -i bash reproduce.sh
```

This will execute the `reproduce.sh` bash script at the root of the repository, which in turn will run the following commands:

1. `python -m src.scripts.get_train_data`. This runs `TSQC` on all the instances in the directories `data/training/biological` and `data/training/ecological`. For each value of k, all values of L from $\mathcal{A} = \{500, 1000, 5000\}$ are used and the optimal one L^* is stored along with the node feature matrix $\mathbf{X}$. Each $(\mathbf{X}, \tilde{L})$ pair is written to a JSONL file in the directory `data/training/`

2. `python -m src.scripts.train_gnn`. This script trains the GNN by first optimizing the hyperparamers and then retraining the model on both the train and validation data. It writes the the trained model parameters and optimal hyperparameters to `results/gnn`. Additionally, it stores the evaluation on the test holdout in `results/gnn/metrics.json`

3. `python -m src.scripts.get_json_results`. This script runs the experiments exactly as outlined in Section 6, and writes the JSON results to both the `results/dimacs` directory and the `results/real-life` directory.

4. `pythom -m src.scripts.get_tables`. This final script reads all the results from the previous scripts, formats them to LaTeX tables, and writes the `.tex` files to the corresponding directories. Specifically, it writes

    - `results/reproducibility.tex`, the LaTeX code for Table 2.
    - `results/dimacs/dimacs.tex`, the LaTeX code for Table 3.
    - `results/real-life/real_life.tex`, the LaTeX code for Table 4.
    - `results/dimacs/dimacs_appendix.tex`, the LaTeX code for Table 6.
    - `results/dimacs/real_life_appendix.tex`, the LaTeX code for Table 7.
    - `results/gnn/hyperparameters.tex`, the LaTeX code for Table 5.

The repository includes a Docker-based development container under `.devcontainer/`. Hence, to prepare the environment, one needs to install Docker Desktop and ensure the Docker daemon is running. Then, in VSCode, simply select *Remote-Containers: Reopen in Container*. This will install all necessary dependencies in `requirements.txt` in the environment.